Project HLM (QML)

Ajmain Adil Tahmid

2026-27

Abstract

A project to build the Quad Model Language

Contents

CHAPTER 1

Introduction

1.1 What is Project HLM?

Project HLM is a project focused on enhancing skills. Its main goal is to create a custom compiled programming language called **QML**.

Fun Fact: Naming of Project HLM

The "HLM" part is completely meaningless. Originally, I took inspiration from Helium (${}^{4}_{2}H$) (HLM → Helium), so I chose .hlm as the file extension, and now I'm just too lazy to rename it.

1.2 What is QML?

QML is an abbreviation for *Quad Model Language*. It is a programming language designed around a four-dimensional design model that captures the core trade-offs present in real-world language and system design.

The fundamental structure of QML is the *quad model*, which defines four primary design goals: **Power**, **Flexibility**, **Connectivity**, and **Precision**. These goals are intentionally placed in tension with one another, reflecting the reality that no programming language can maximize all dimensions simultaneously without compromise. QML is an exploration of control, responsibility, and trade-offs in low-level language design.

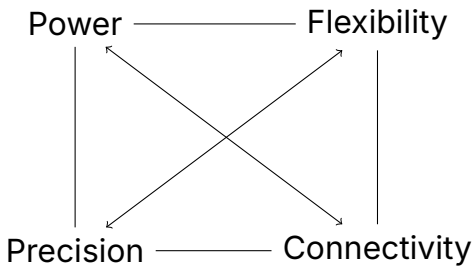

Figure 1.1: The primary quad model illustrating the four core design goals of QML.

Figure 1.1 illustrates the primary quad model of QML and the relationships between these four goals. Each axis represents a dimension in which a language implementation may prioritize certain features over others. Rather than attempting to hide these trade-offs, QML exposes them explicitly as part of its design philosophy.

As a result, QML encourages deliberate and transparent design decisions, allowing developers to reason clearly about the strengths and limitations of their programs.

Note

There are multiple quad-based models within QML, each derived from the same core structure shown in Figure 1.1. This design approach is the origin of the name *Quad Model Language*.

1.3 Achieving the Design Goals of QML

QML is a low- to mid-level programming language, conceptually similar to C++, designed to give maximum control to the programmer while remaining concise and practical. Its design goals are achieved not through abstraction-heavy features, but through deliberate exposure of system-level capabilities and explicit control mechanisms.

Power

In QML, power refers to the degree of control granted to the programmer. The language is designed so that the user, rather than the runtime or language constraints, determines how software behaves. QML allows direct access to low-level operations, enabling the development of highly powerful software without artificial limitations imposed by the language itself. Whether the target is a performance-critical engine or a complex system application, QML does not restrict what can be expressed or implemented.

Flexibility

QML achieves flexibility by being applicable across a wide range of domains. The language can be used for tasks ranging from low-level systems programming, such as physics engines and performance-critical software, to higher-level automation tasks, including interaction with third-party applications. This breadth is made possible by QML's concise syntax, general-purpose design, and lack of domain-specific constraints, allowing the same language to adapt to multiple problem spaces.

Connectivity

Connectivity in QML is achieved through built-in support for interoperability with external codebases, particularly C++. QML allows seamless integration of C++ code, enabling developers to reuse existing libraries and systems without rewriting them. This design choice ensures that QML can function both as a standalone language and as a bridge between components written in other languages.

Precision

Precision in QML is enforced through explicit semantics and low-level control, including direct memory manipulation. Rather than hiding memory behavior behind abstractions, QML allows programmers to manage memory deliberately and predictably. This approach reduces ambiguity in program execution and ensures that behavior is consistent, deterministic, and transparent to the developer.

1.3.1 Intentional Trade-offs

QML acknowledges that providing greater control and flexibility often increases responsibility for the programmer. By exposing low-level mechanisms such as memory management and external code integration, QML prioritizes precision and power over enforced safety. These trade-offs are intentional and reflect QML's goal of empowering the programmer rather than constraining them.

Fun Fact:

QML assumes that the user is a fully responsible programmer. If a QML program crashes, corrupts memory, or brings down the system, the language considers this a consequence of deliberate control rather than a failure of protection.

Note

Although QML shares similarities with C++ in terms of control and performance, it is not intended to replace C++. Instead, QML is designed to coexist with C++ by allowing direct integration of C++ code, combining familiar low-level power with QML's own language structure.

1.4 Constraints, Resources, and Context

QML is developed as a personal research project under practical constraints related to resources, time, and environment. These constraints are an integral part of the project's context rather than limitations of intent.

Development is conducted on a HP EliteBook 840 G4 laptop equipped with a 7th generation Intel Core i5 processor. This system represents the most capable hardware reasonably available to the author without external funding. Rather than prioritizing hardware upgrades, the project emphasizes efficient design, careful experimentation, and a deep understanding of underlying systems.

Time availability is also constrained by academic responsibilities. As a secondary school student preparing for the SSC examination in Bangladesh, development time is limited and must be balanced with formal education. Progress on QML therefore occurs incrementally, guided by long-term research goals rather than short-term feature completion.

These constraints have shaped QML's development philosophy. The project prioritizes conceptual clarity, explicit design trade-offs, and foundational correctness over rapid expansion. Additional time, resources, or funding would enable deeper exploration of compiler infrastructure, formal semantics, and advanced tooling, accelerating the project without altering its core vision. External support or mentorship would primarily accelerate experimentation rather than redefine the project's direction.

CHAPTER 2

Grammar

This chapter defines the core grammatical structure of QML. The grammar is specified using a combination of formal notation and Extended Backus–Naur Form (EBNF) to ensure clarity and precision.

2.1 Return Statement

In QML, every program and every function is required to terminate by returning a value. This requirement enforces explicit control flow and avoids implicit program termination.

Note

As a best practice, the `main` function is expected to return an integer value, following conventions common to low-level and systems programming languages.

Formally, a return statement consists of the `return` keyword followed by a single expression:

$$\text{return} \rightarrow \text{expression}$$

The return statement accepts exactly one expression and must be terminated with a semicolon. An example is shown below:

```
return 0;
```

The corresponding EBNF definition is:

```
return_stmt = "return", expression, ";" ;
```

2.2 Variable Declaration

Variables in QML must be declared with an explicit data type. Implicit typing is not supported, ensuring predictable memory layout and behavior.

The general form of a variable declaration is:

$$\text{dataType identifier} = \text{expression};$$

The corresponding EBNF definition is:

```
variable_declearing_stmt = DATA_TYPE, identifier, "=", expression, ";" ;
```

2.3 Data Types

QML currently supports a set of primitive numeric data types. These types are designed to provide explicit control over size and precision, similar to low-level languages such as C and C++.

The supported numeric types are:

- `int`
- `float`

These types are both `signed` and `unsigned`. These types can be used wherever an expression is expected. Additional data types may be introduced in future iterations of the language.

The full list of

2.3.1 Numbers

Integer

Long Numbers

Floating Numbers

Double

2.3.2 Characters

2.3.3 Boolean

2.3.4 Arrays

2.3.5 Objects

2.3.6 User-Defined Types (Structures)

2.4 Functions